

Intersection Traffic Simulator

Helen N. & Xin Wang





Overview

About the Project and High-Level Block Diagram



Project Description

1

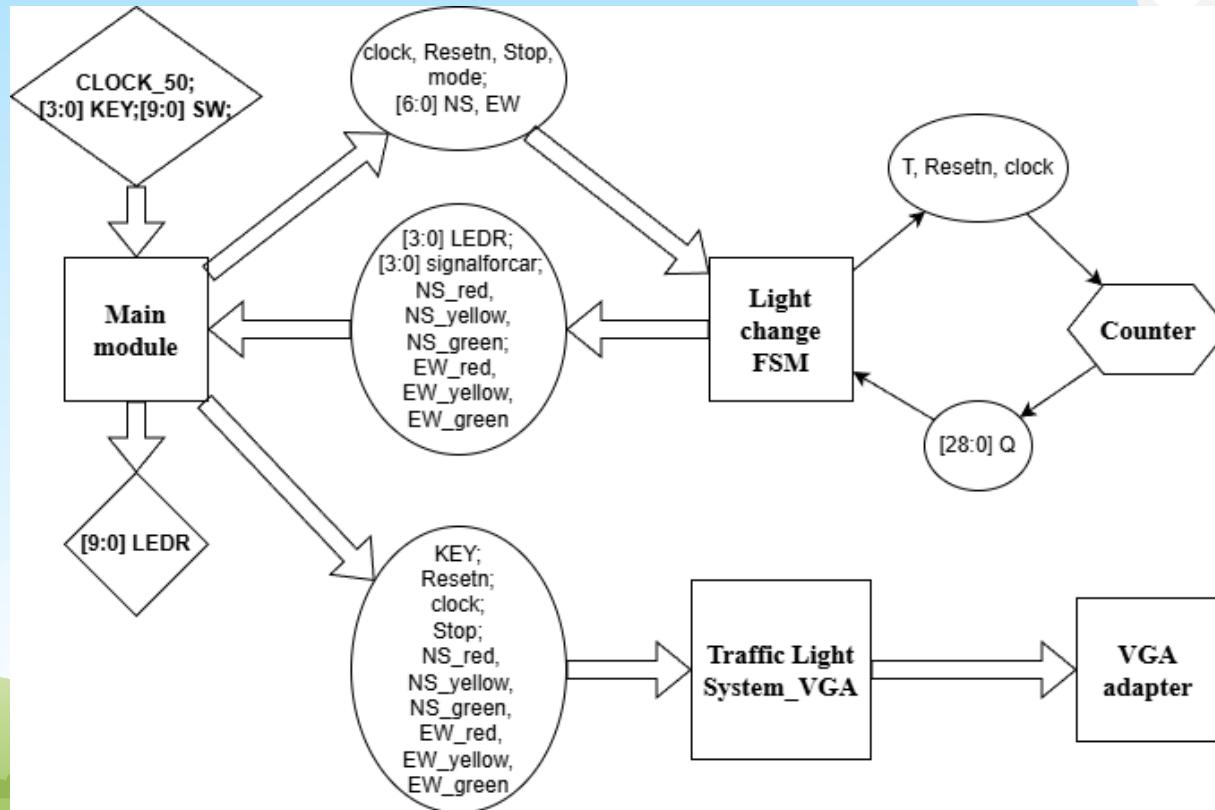
Simulate the Traffic Flow at Intersection

2

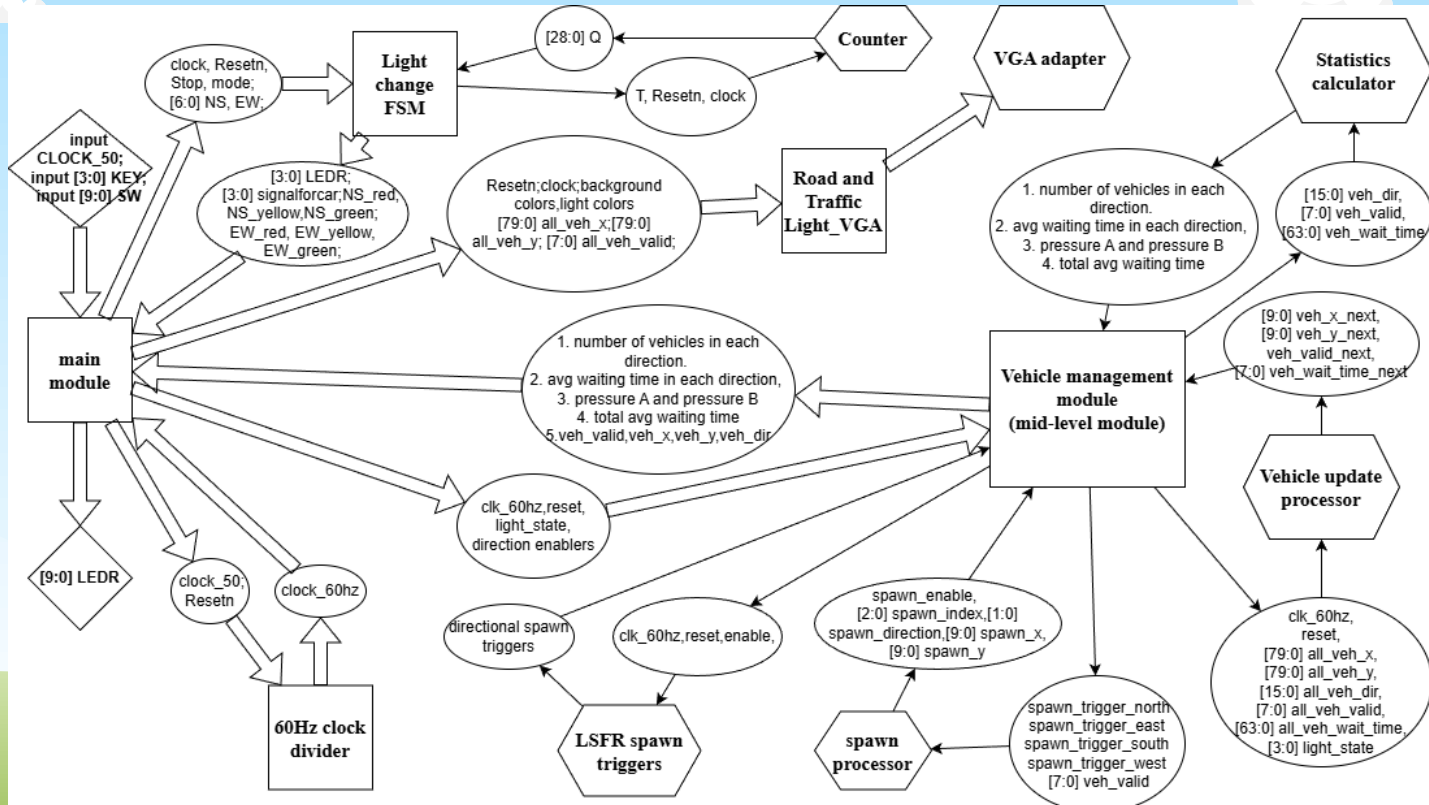
Purpose

This simulation serves as a foundation for testing future intelligent systems, such as the automated sensor-based traffic lights, by providing a controlled environment to measure efficiency improvements

Project Block Diagram



Project Block Diagram



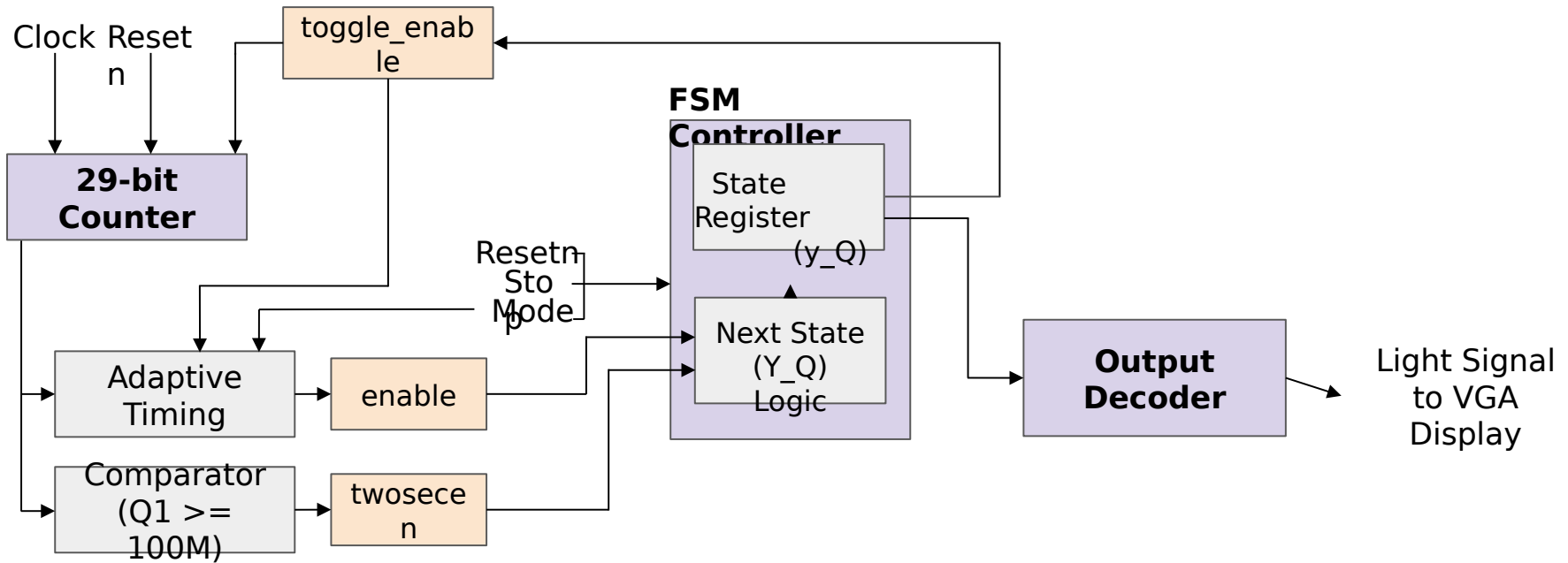


Baseline Mode

Project Display with Repeated Pattern
Car



Light Change FSM



State Table

State	NS Lights	EW Lights	Time Duration	Timing Control
A	RED	YELLOW	2 seconds	twosecen
B	RED	RED	2 seconds	twosecen
C	GREEN	RED	7 seconds	enable
D	YELLOW	RED	2 seconds	twosecen
E	RED	RED	2 seconds	twosecen
F	RED	GREEN	7 seconds	enable

Bug Encountered

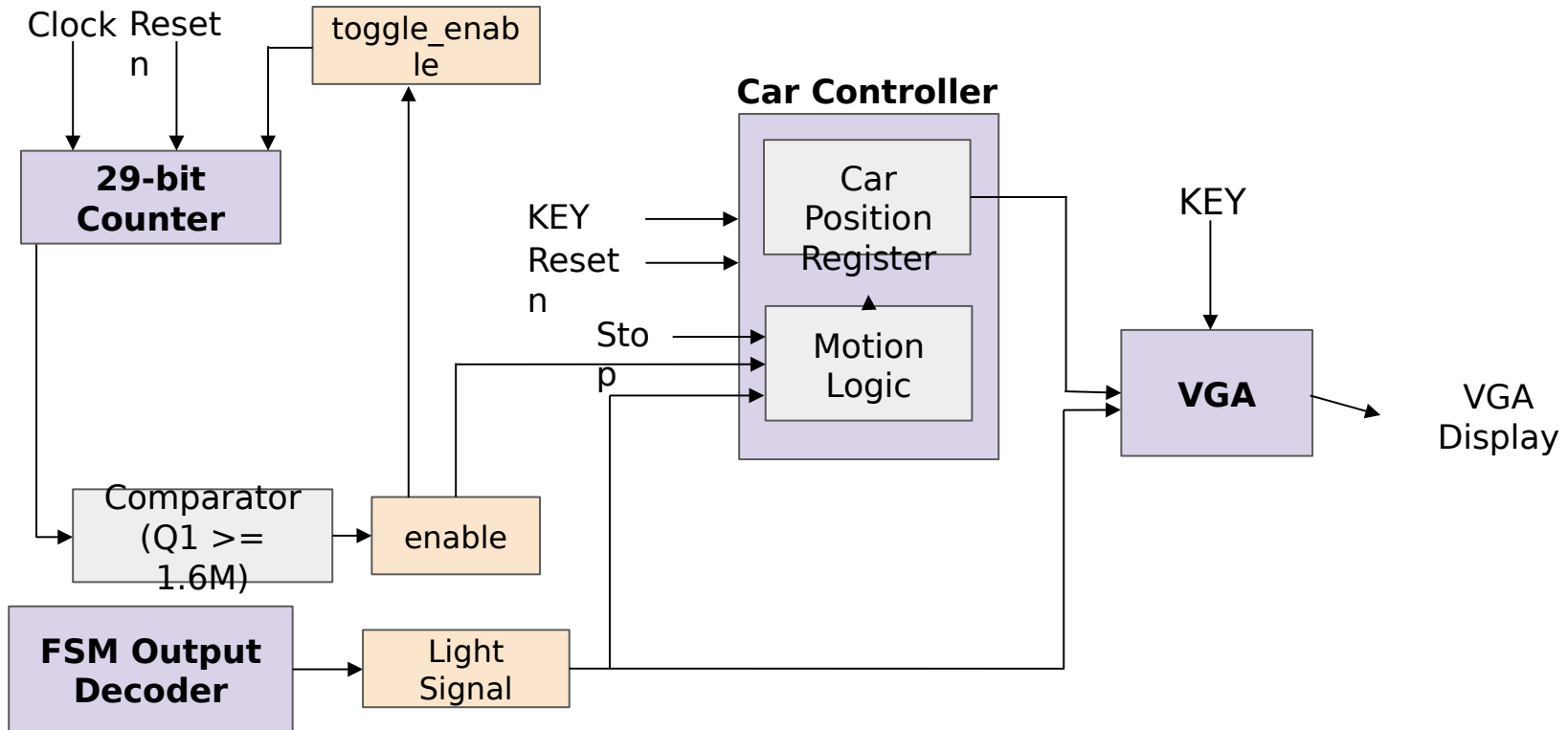
**The Light
Change FSM
skips through
two second
light duration**



**Introduce Toggle
Enable into
State Register
and Modified
Counter**

To control the counter
and reset the counter
when changing to the
next state and
resetting

VGA Display



Car Behavior State

Resetn	KEY	Stop	Light State	Car Behavior
0	X	X	X	Reset to initial position
1	0	X	X	Reset to initial position
1	1	1	X	Remain in previous position
1	1	0	GREEN	Move forward
1	1	0	YELLOW / RED	Stop before Line / Move until the Line



DEMO!

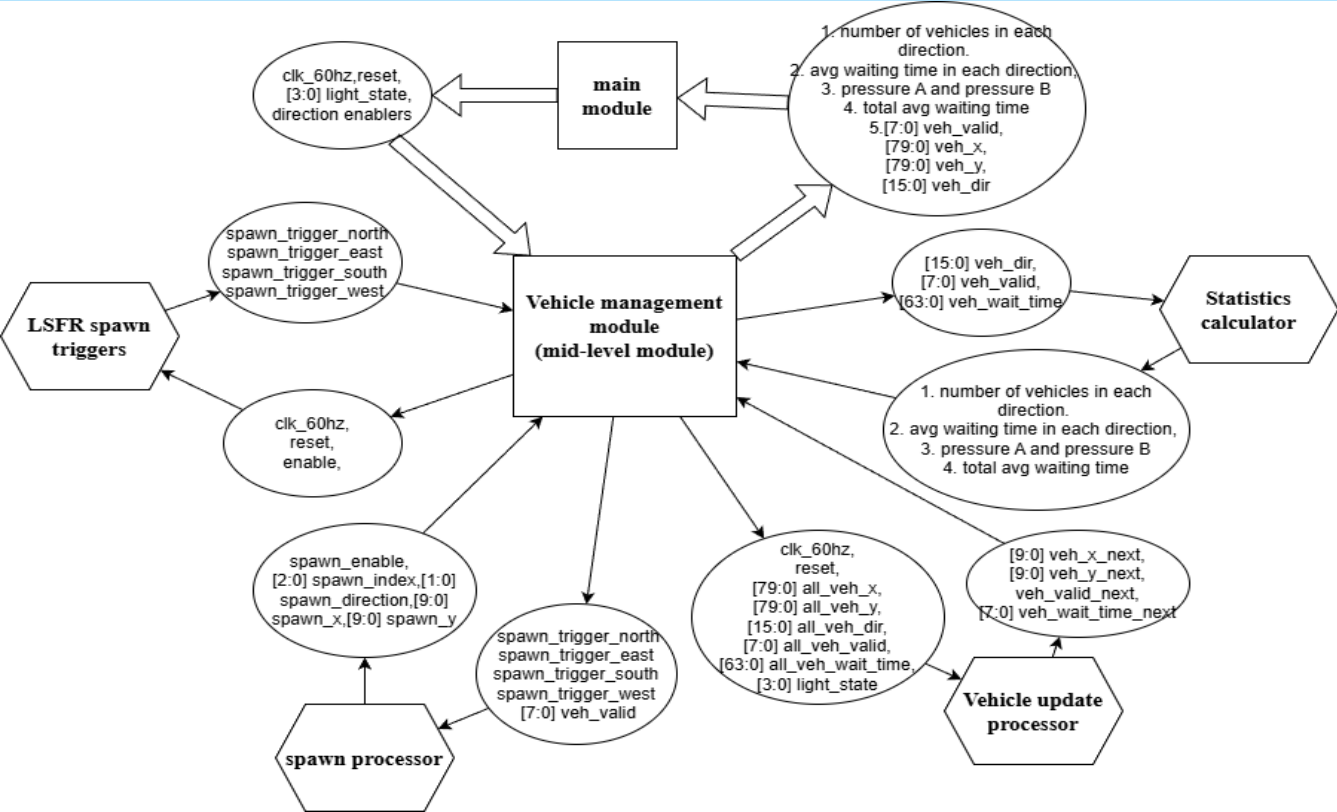


Vehicle Logic

Randomized Input Car



Vehicle Management



Bugs Encountered

1

Boundary Check: Coordinates Wrap Around

```
// 6. boundary check
always @(*)
begin
    out_of_bounds = 0;

    if (veh_valid)
    begin
        case (veh_dir)
            DIR_NORTH:
            begin
                if (veh_y_next > 10'd480)
                begin
                    out_of_bounds = 1;
                end
            end

            DIR_EAST:
            begin
                if (veh_x == 10'd0 || veh_x_next > veh_x) // x may warp around
                begin
                    out_of_bounds = 1;
                end
            end

            DIR_SOUTH:
            begin
                if (veh_y == 10'd0 || veh_y_next > veh_y) //y may wrap around
                begin
                    out_of_bounds = 1;
                end
            end
        end
    end
end
```

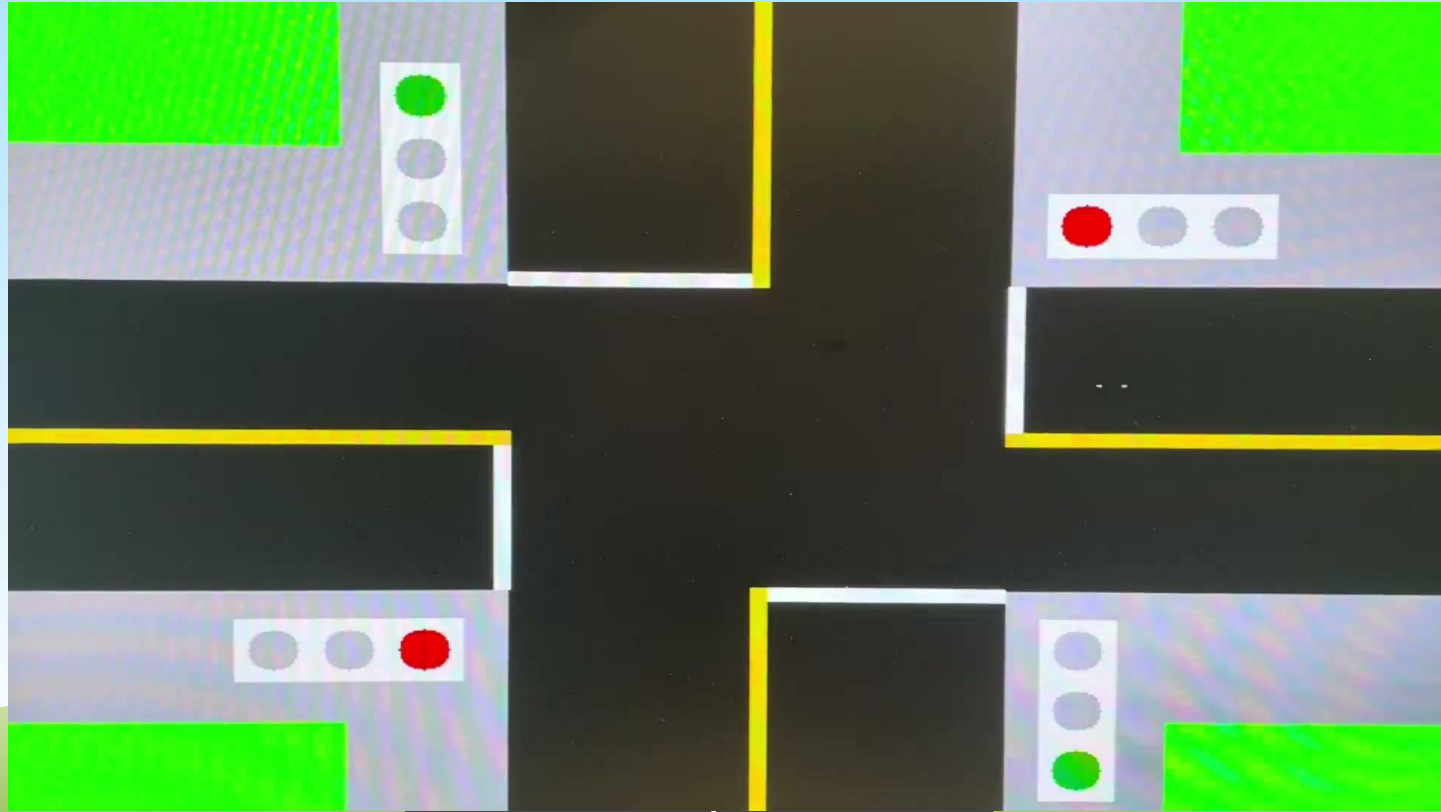
Bugs Encountered

```
// spawn outputs
output reg spawn_enable,
output reg [2:0] spawn_index, /
output reg [1:0] spawn_direction, /
output reg [9:0] spawn_x, /
output reg [9:0] spawn_y
```

2

**Modular Design:
Arrays are modified in
submodules**

Vehicle Logic Display



Future Development

1

Incorporate the vehicle logic into the simulation

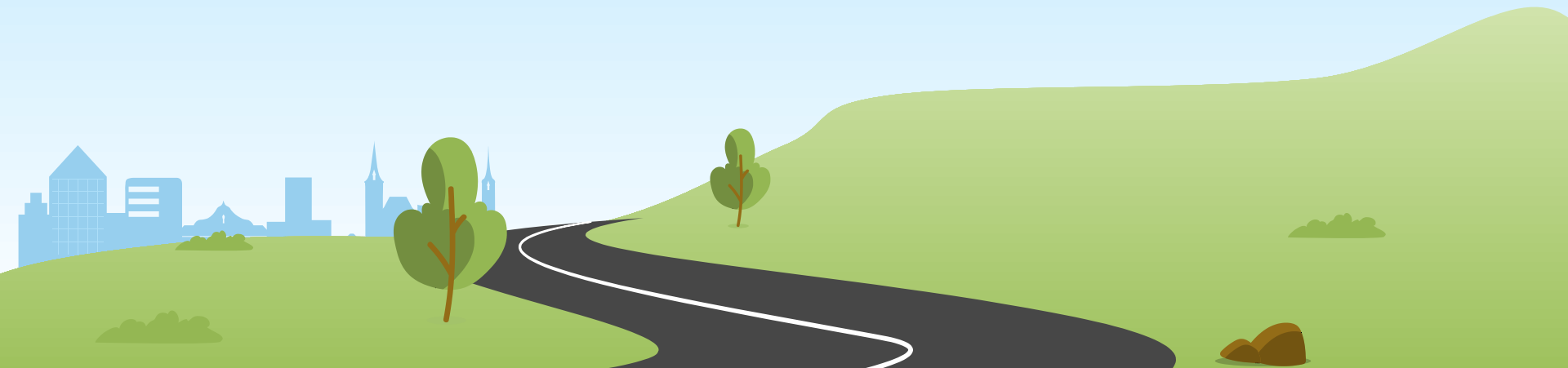
2

Implement pressure based algorithm

3

Displaying data (Max Wait Time, Average Wait Time) after pressing "Stop" switch

**Thank you
for listening!**



Work Division

Helen N.	Xin Wang
Modified Counter	Vehicle management
Light Change FSM	LFSR spawn trigger
VGA for road and traffic light	Spawn processor
VGA for repeated pattern car	Vehicle update processor
Green Light logic	Statistics calculator
	VGA for the dotted cars
	Clock Divider

References

- ChipVerify. (n.d.). Verilog parameters. ChipVerify.
<https://www.chipverify.com/verilog/verilog-parameters>
- ChipVerify. (n.d.). SystemVerilog packed arrays. ChipVerify.
<https://www.chipverify.com/systemverilog/systemverilog-packed-arrays>
- Linear-feedback shift register. (2025). In Wikipedia.
https://en.wikipedia.org/wiki/Linear-feedback_shift_register
- Xeroxpop. (2017, January 5). Bit slicing in Verilog [Online forum post]. Stack Overflow.
<https://stackoverflow.com/questions/41484219/bit-slicing-in-verilog>

Appendix A: Baseline Demo

